

AGENTS.md

rodaduaAI Development Rules

This document defines mandatory rules for AI coding agents working on rodaduaAI.

Project Overview

rodaduaAI is an AI-powered motorcycle assistant focused on Indonesian riders.

Primary goals:

- Motorcycle troubleshooting
 - Maintenance recommendations
 - Spare part recommendations
 - Motorcycle specifications
 - Motorcycle comparison
 - Workshop recommendations
-

Technology Stack

Backend

- Laravel 12
- PHP 8.4+
- MySQL 8
- Redis
- Laravel Queue
- Laravel Horizon
- Laravel Sanctum

Frontend

- Vue 3
 - InertiaJS
 - Vite
 - TailwindCSS
 - Pinia
 - TypeScript
-

General Principles

Rule 1

Always prioritize:

1. Maintainability
2. Readability
3. Security
4. Performance

Never sacrifice maintainability for short-term convenience.

Rule 2

Before implementing any feature:

- Understand existing architecture
 - Reuse existing patterns
 - Avoid introducing new patterns unless necessary
-

Rule 3

Avoid premature optimization.

Prefer simple solutions first.

Architecture Rules

Backend Architecture

Controllers must remain thin.

Allowed:

```
public function store(StoreMotorcycleRequest $request)
{
    return $this->service->create(
        CreateMotorcycleDTO::fromRequest($request)
    );
}
```

```
    );  
}
```

Not allowed:

```
public function store(Request $request)  
{  
    // 200+ lines business logic  
}
```

Layer Structure

Feature flow:

Controller → DTO → Service → Repository → Model

Business logic belongs inside Services.

Complex queries belong inside Repositories.

Validation belongs inside Form Requests.

Folder Structure

```
app/  
├─ Actions  
├─ DTOs  
├─ Services  
├─ Repositories  
├─ Models  
├─ Jobs  
├─ Events  
├─ Policies  
├─ Exceptions  
└─ Http
```

Frontend Architecture

Component Rules

Components must be small and reusable.

Target:

- Under 200 lines preferred
- Under 300 lines maximum

Large components must be split.

Data Fetching

Never place API logic directly inside UI components.

Use:

```
Services/  
Composables/  
Stores/
```

Example:

```
MotorcycleService.getList()
```

Not:

```
axios.get(...)
```

inside page components.

State Management

Global state:

- Pinia

Local state:

- ref()
- reactive()

Do not duplicate state.

TypeScript Rules

Never use:

any

Prefer:

```
interface  
type  
generics
```

Bad:

```
const data: any
```

Good:

```
interface Motorcycle {  
  id: number  
  name: string  
}
```

Design System Rules

Colors

Use design tokens only.

Never hardcode colors.

Bad:

```
class="bg-red-500"
```

Good:

```
class="bg-primary"
```

Spacing

Use consistent spacing scale.

Allowed:

```
4  
8  
12  
16  
24  
32  
48  
64
```

Avoid arbitrary values.

Typography

Use predefined typography tokens.

Never manually adjust font sizes per page.

Database Rules

Migrations

Always:

- Add indexes where appropriate
- Use foreign keys
- Use cascade rules carefully

Example:

```
$table->foreignId('user_id')
    ->constrained()
    ->cascadeOnDelete();
```

Queries

Never use N+1 queries.

Always evaluate:

```
with()
load()
loadMissing()
```

when retrieving relationships.

API Standards

Success Response

```
{
  "success": true,
  "data": {}
}
```

Error Response

```
{  
  "success": false,  
  "message": "Validation failed",  
  "errors": {}  
}
```

Keep response format consistent.

Security Rules

Authentication

Use Laravel Sanctum.

Never create custom authentication systems.

Authorization

Use:

- Policies
- Gates

Never trust frontend permissions.

Validation

Every input must be validated.

No exceptions.

Use Form Requests.

Mass Assignment

Always define:


```
$fillable
```

or

```
$guarded
```

Never leave models unprotected.

Secrets

Never hardcode:

- API keys
- Passwords
- Tokens
- Secrets

Always use:

```
.env  
config()
```

AI Layer Rules

Provider Isolation

Never call OpenAI directly from controllers.

Always use:

```
AIProviderInterface
```

Example:

```
OpenAIProvider  
DeepSeekProvider
```

Prompt Management

Prompts must be versioned.

Store prompts separately from business logic.

Avoid prompt strings scattered across codebase.

AI Safety

Validate:

- User input
- AI output

Never trust AI-generated content.

Queue Rules

Queue all expensive tasks:

- AI requests
- OCR processing
- Report generation
- Data imports

Do not block HTTP requests.

Testing Rules

Every feature must include tests.

Minimum:

Backend

- Feature Test

Preferred:

- Unit Test

- Feature Test
-

Frontend

Critical flows require:

- Component Tests
 - E2E Tests
-

Technical Debt Rules

Before introducing a dependency:

Ask:

1. Can Laravel already do this?
2. Can Vue already do this?
3. Can existing code solve this?

If yes:

Do not add dependency.

Performance Rules

Backend

Prefer:

- eager loading
- caching
- queues

Avoid:

- repeated queries
 - large loops with queries
-

Frontend

Use:

- lazy loading
- code splitting
- dynamic imports

For large pages.

Git Rules

Commit messages:

```
feat(auth): add login page  
  
feat(ai): add prompt versioning  
  
fix(chat): resolve session timeout  
  
refactor(user): simplify repository  
  
test(ai): add provider tests
```

Definition of Done

A task is complete only when:

- Feature implemented
- Validation implemented
- Authorization implemented
- Tests implemented
- Types added
- No lint errors
- No static analysis errors
- Documentation updated

Otherwise the task is not done.

Agent Behavior

When generating code:

1. Follow existing project conventions.
2. Prefer modifying existing code over creating new abstractions.
3. Keep implementations simple.
4. Avoid overengineering.
5. Explain architectural decisions when introducing new patterns.
6. Reject solutions that create unnecessary technical debt.
7. Prioritize security and maintainability.
8. Generate production-ready code only.

The objective is to build a maintainable MVP that can scale into Indonesia's leading AI motorcycle assistant platform.